

Safe On Route

José Adriel Marcos
Raúl Cristian Dumitru Tanase





ÍNDICE

ÍNDICE	2
Creación y configuración de instancia en Lightsail	3
Replicar el servicio en una instancia propia	3
Case sensitive en producción	3
PM2: Gestor de procesos	3
Firewall y puertos	4
Reverse proxy	4
Comandos útiles de administración	4



Creación y configuración de instancia en Lightsail

Lightsail es un servicio de AWS que ofrece servidores virtualizados privados (VPS) simplificados, ideal para proyectos pequeños. Mucho más sencillo e infinitamente más barato que EC2. El plan elegido en este caso ha sido una VPS de 1Gb de RAM y 2VCpus de 7\$/mo. Los primeros 90 días han sido gratuitos. La VPS posee el S.O Debian (Linux).

Otra de las decisiones tomadas en base a la magnitud del proyecto es el deploy de backend y frontend en una sola instancia dado que no va a requerir de división por recursos. Sin embargo, la base de datos es cierto que está alojada de forma externa en piensa solutions.

Replicar el servicio en una instancia propia

Para poder replicar este proyecto en otra instancia diferente han de instalarse paquetes obligatoriamente como; Nodejs, git, pm2 y Nginx.

Para realizar el setup del servicio en una instancia propia:

1. Clonar el repositorio con el comando git clone.
2. En los directorios /server y /client se debe ejecutar el comando npm install.
3. Se debe crear el archivo .env en /server con las variables de entorno del env.example.
4. Se debe ejecutar el comando npm run build en /client.

Case sensitive en producción

En un principio durante el desarrollo en windows no hubo ningun problema con los nombres de los archivos porque windows no es case sensitive. Sin embargo, al desplegar en linux el servidor no arrancaba dado que si es case sensitive. El problema concreto sucedio con el archivo realDataSequelizeRepository que se importaba con la primera en mayúscula dando el problema MODULE NOT FOUND en linux.

PM2: Gestor de procesos

PM2 es un gestor de procesos para Node.js en producción. Sin este gestor, si la app se cae o se reinicia la instancia tendria que arrancarse todo manualmente otra vez. Además, la VPS no podría ser usada para otras funciones. Por lo tanto, mantiene toda la app en segundo plano.

Comandos utilizados:

- pm2 start src/index.js --name safe-on-route → Arranca la app con un nombre.
- pm2 startup → Genera un script para que se arranque el servidor al reiniciar el servidor.
- pm2 save → Guarda la lista de procesos para que se restauren tras un reinicio.
- pm2 status → Ver estado.
- pm2 logs → Ver logs en tiempo real.
- pm2 restart safe-on-route → Reiniciar la app.



Firewall y puertos

En Lightsail el networking se gestiona desde el propio panel web de AWS. Por defecto únicamente viene abierto el puerto 22 (SSH), por ende hay que abrir el 80 (HTTP), 443 (HTTPS) y 3000 (puerto de la app que se podría desactivar una vez configurado el Nginx).

Reverse proxy

Un reverse proxy es un servidor mediante el cual un usuario accede por el puerto 80 (HTTP) y es automáticamente redirigido al puerto 3000 (donde corre la app). Por lo que introduciendo la URL en el navegador no haría falta poner “:3000” al final.

Para configurarlo simplemente hay que modificar el archivo `/etc/nginx/sites-available/default` indicando el proxy pass.

En este caso surgió un problema dado que el puerto 80 venía activado por defecto y fue ocupado por node en lugar de por Nginx. Por lo tanto se tuvo que hardkillar el proceso dejándole paso a Nginx.

Comandos útiles de administración

- `pm2 start src/index.js --name safe-on-route` → Arranca la app con un nombre.
- `pm2 startup` → Genera un script para que se arranque el servidor al reiniciar el servidor.
- `pm2 save` → Guarda la lista de procesos para que se restauren tras un reinicio.
- `pm2 status` → Ver estado.
- `pm2 logs` → Ver logs en tiempo real.
- `pm2 restart safe-on-route` → Reiniciar la app.
- `sudo systemctl restart nginx` → Reiniciar Nginx.
- `sudo systemctl status nginx` → Ver estado de Nginx.
- `sudo nginx -t` → Testear config de Nginx antes de reiniciar.
- `sudo fuser -k 80/tcp` → Mata cualquier proceso en el puerto 80.
- `git pull` → Actualiza el código desde github.
- `npm run build` → Regenera el frontend despues de un pull.